

Comp 484 – HW10

1. Message Logging

Log Info

`console.info()` is used to log a general status message when the page first loads, and again when the user renames the pet. This was placed at the top of the document ready function to confirm the page initialized correctly.

```
>> console.info('Gigapet loaded successfully')
i Gigapet loaded successfully debugger eval code:1:9
```

Log Warning

`console.warn()` is placed inside `checkWeightAndHappinessBeforeUpdating()`. Whenever a stat like weight or happiness hits zero, a yellow warning message fires in the console. This is useful during testing because it flags edge cases without breaking anything or stopping execution.

```
Exercise done – happiness: -2, weight: -2 script.js:88:13
⚠ Weight hit zero – further exercise or play will have no effect on weight script.js:111:17
```

Log Error

`console.error()` is used inside a try/catch block on page load. An undefined variable is intentionally called with `.toString()`, which throws a `TypeError`. The catch block intercepts it and passes the error message to `console.error()`, which prints it in red.

```
Gigapet is running script.js:34:13
! ▶ TypeError caught: can't access property "toString", badValue is undefined script.js:45:17
```

Log Table

`console.table()` is called right after the `pet_info` object is initialized. It displays all four properties (name, weight, happiness, cleanliness) in a formatted table directly in the console. This makes it easy to verify the starting state

```
>> console.table(pet_info)
console.table() debugger eval code:1:9
```

(index)	Values
name	Squirtle
weight	0
happiness	0
cleanliness	5

Log Group

`console.group()` and `console.groupEnd()` are used to wrap the two startup messages under a collapsible label called "Startup". This keeps the console organized when multiple messages fire at once on page load.

```
>> console.group('Startup');
▼ Startup debugger eval code:1:9
  >> console.log('Buttons registered');
    Buttons registered debugger eval code:1:9
  < undefined
  >> console.log('Buttons registered');
    Buttons registered debugger eval code:1:9
  < undefined
  >> console.log('DOM ready');
    DOM ready debugger eval code:1:9
  < undefined
  >> console.groupEnd();
  < undefined
```

Log Custom

A styled message is logged using the %c directive inside console.log(). A CSS string is passed as the second argument, which applies blue bold text at a larger font size to the message in the console. This is useful for making important status messages stand out among other output.

```
>> console.log('%cGigapet is running', 'color: #3a86ff; font-weight: bold; font-size: 14px;');
  Gigapet is running debugger eval code:1:9
```

2. View Messages Logged by the Browser

Cause a 404 Network Error

A new Image object is created on page load with a src path pointing to a file that does not exist. The browser automatically tries to fetch it, fails, and logs a 404 error in both the Console and the Network tab without any extra code needed. This shows how the browser surfaces network errors on its own.

```
▶ GET http://127.0.0.1:5500/images/does_not_exist.png [HTTP/1.1 404 Not Found 3ms]
```

Cause a TypeError

A TypeError is triggered by calling .toString() on an undefined variable inside a try/catch block. Before the catch was added, this would appear in the console as an uncaught browser error. The catch block intercepts it and passes it to console.error(), giving control over how the error is displayed.

```
Gigapet is running script.js:34:13
❗ ▶ TypeError caught: can't access property "toString", badValue is undefined script.js:45:17
```

Cause a Violation

A forced layout violation is triggered by reading the offsetWidth property of a DOM element inside a loop 100 times on page load. Each read forces the browser to recalculate layout before the loop finishes, which Chrome DevTools flags as a “Forced reflow” violation in the console. To see it, open the Console, make sure Violations are enabled in the log level dropdown, then refresh the page. The message appears automatically without any button clicks.

3. Filter Messages

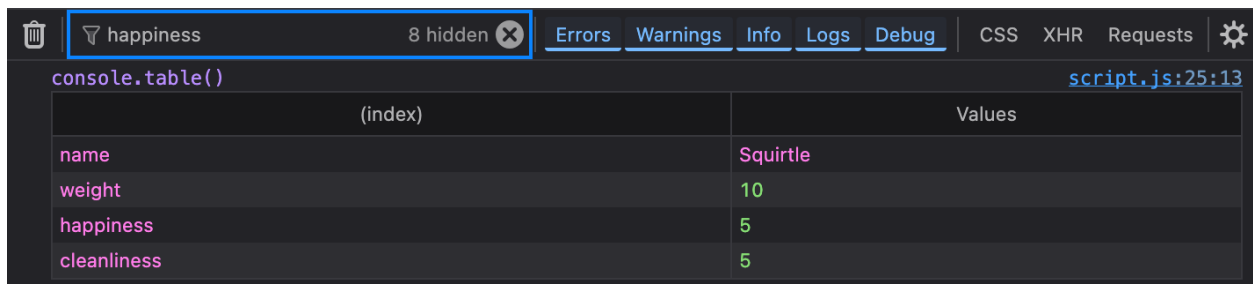
Filter by Log Level

Clicking the log level dropdown in the Console filter bar lets you show only specific message types. Selecting "Warnings" hides everything except the stat-floor warnings that fire from `checkWeightAndHappinessBeforeUpdating()` when weight or happiness hits zero. This makes it easy to focus on one category of output at a time.



Filter by Text

Typing a keyword like "happiness" into the console filter bar instantly hides all messages that do not contain that word. This is useful for isolating just the logs that fire when the happiness stat changes, like the ones inside `clickedTreatButton()` and `clickedPlayButton()`.

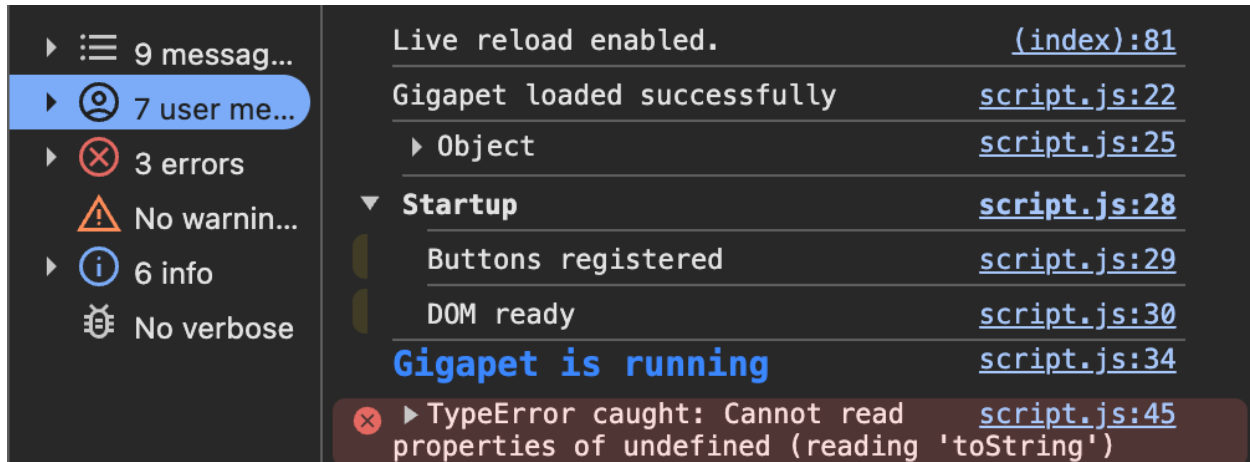


Filter by Regular Expression

To filter by regular expression, open the Console and find the filter bar at the top. Click the small `.*` button to the left of the input to enable regex mode. Then type a pattern like `/weight|cleanliness/` into the bar. The console will only show messages matching either word. To generate results, click Exercise several times until weight hits zero and Bath until cleanliness warnings appear. Only those matching messages will be visible while the filter is active.

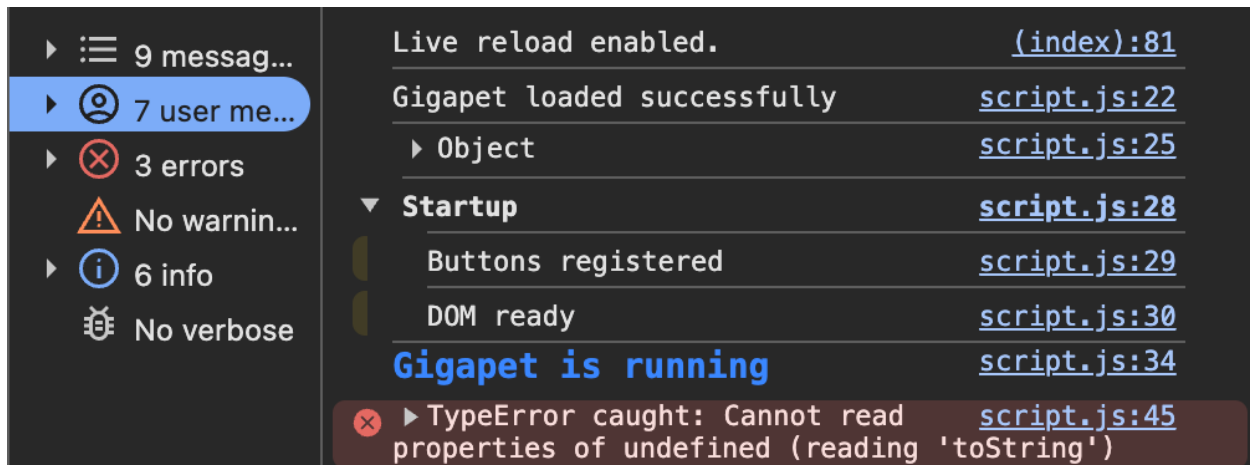
Filter by Message Source

The Sidebar panel in DevTools groups messages by source file. Opening it and selecting `script.js` under the user messages section shows only the logs written in our own code, hiding anything generated by jQuery or the browser itself. This makes it easy to focus on just the output we added.



Filter by User Messages

Selecting "user messages" in the DevTools Console sidebar filters the output to show only messages that were explicitly written in our JavaScript code. This excludes anything the browser or third-party libraries like jQuery generated automatically, leaving just the logs from `script.js`.



4. Reproduce a Bug

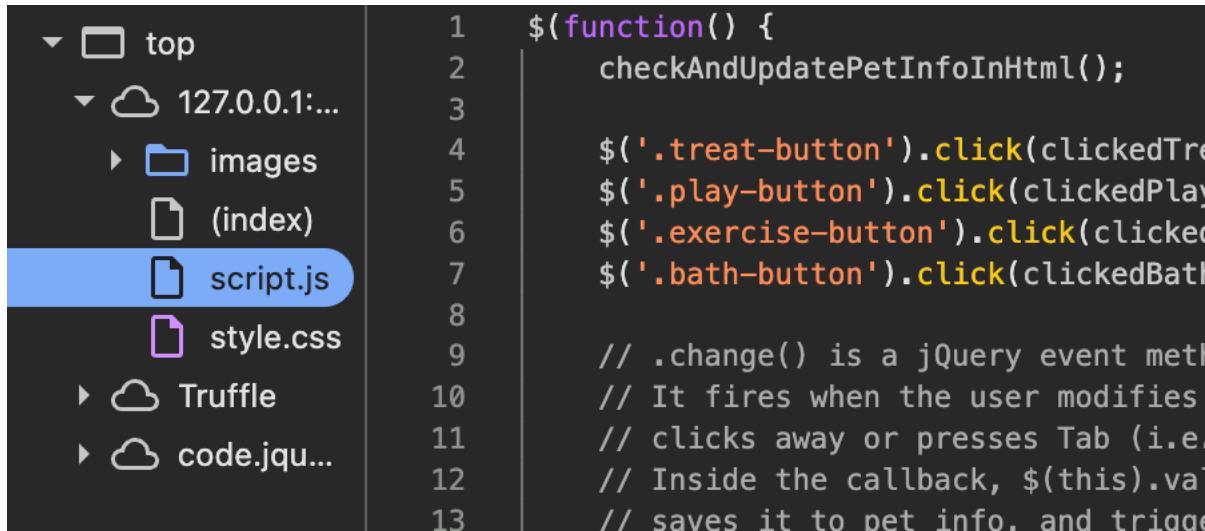
An earlier version of the project had a bug where clicking a button multiple times would stop showing the speech bubble notification. The root cause was that the `.notification-template` div lived inside `#speech-area`, so the first call to `.empty()` deleted the template. Every subsequent `.clone()` had nothing to copy, so no bubble appeared.

To reproduce: place the template div back inside `#speech-area`, load the page, click Treat, wait for the bubble to fade, then click Treat again. The second click produces no bubble.

The fix was to move `.notification-template` outside of `#speech-area` in the HTML so `.empty()` never touches it. After that change, every button click reliably produces a fresh bubble.

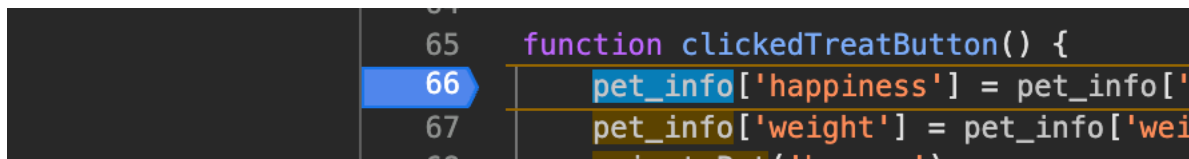
5. Get Familiar with the Sources UI

Opening the Sources panel in DevTools (F12 then Sources) shows the full project file tree on the left side. Clicking on `script.js` loads the complete source file with line numbers in the center panel. From here you can read through the code, set breakpoints by clicking line numbers, and navigate between functions.



6. Pause the Code with a Breakpoint

A breakpoint was set by clicking the line number on the first line inside `clickedTreatButton()` in the Sources panel. After clicking the Treat button on the page, execution paused at that line before any stat changes were applied. The current call stack and variable values are visible on the right side while paused.



7. Set a Line-of-Code Breakpoint

A line-of-code breakpoint was placed directly on the conditional inside `checkWeightAndHappinessBeforeUpdating()` where weight is clamped to zero. To trigger it, the Exercise button was clicked enough times to bring weight down to zero. Execution paused on that exact line, allowing inspection of the variable state right before the clamp ran.

```

106 function checkWeightAndHappinessBeforeUpdating() {
107     // Prevent stats from going below zero
108     if (pet_info['weight'] < 0) {
109         pet_info['weight'] = 0;
110         // Log Warning: stat hit the floor, something to keep track of
111         console.warn('Weight hit zero - further exercise or petting needed');
112     }
113     if (pet_info['happiness'] < 0) {
114         pet_info['happiness'] = 0;
115         console.warn('Happiness hit zero - pet is not happy');
116     }
117     if (pet_info['cleanliness'] < 0) {

```

8. Check Variable Values

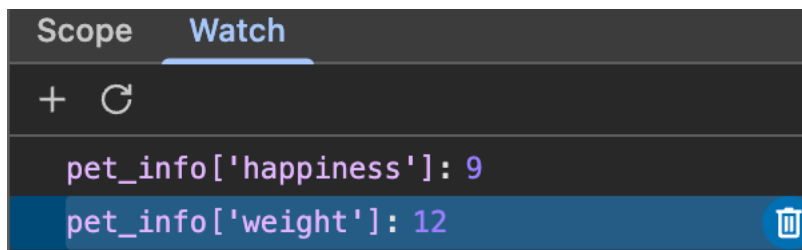
The Scope Pane

With execution paused at a breakpoint, the Scope pane on the right side of the Sources panel showed all current variable values. The `pet_info` object was listed under Global scope with all four properties and their values at the moment of the pause. This makes it easy to see the exact state of the program without adding any extra logging.

The screenshot shows a code editor with a breakpoint set at line 108, column 5, in the file `script.js`. The code is paused at the `if (pet_info['weight'] < 0) {` statement. The Scope pane on the right shows the current scope, which is `Local`. Under `this: Window`, various global objects are listed, including `$`, `alert`, `animatePet`, `atob`, `blur`, `btoa`, `caches`, `cancelAnimationFrame`, `cancelIdleCallback`, and `captureEvents`. The Call Stack on the left shows the function `checkWeightAndHappinessBeforeUpdating` at `script.js:108`.

Watch Expressions

Watch expressions were added for `pet_info['happiness']` and `pet_info['weight']` by clicking the `+` icon in the Watch pane. These values update automatically each time execution pauses, making it easy to track specific stats across multiple button clicks without having to expand the full scope tree each time.



The Console

While paused at a breakpoint, the console is still active and has access to all variables in the current scope. Typing `pet_info` into the console and pressing Enter returned the full object with all four properties. Typing `pet_info["happiness"]` returned just the happiness value. This is a quick way to inspect or test values without modifying the code.



9. Apply a Fix

While paused inside `clickedExerciseButton()`, the Scope pane showed happiness going negative after the subtraction ran but before the bounds check fired. This confirmed that `checkWeightAndHappinessBeforeUpdating()` was the right place to apply the fix. Stepping through with the breakpoint active showed the clamp conditional running correctly and the value being set back to zero before it was ever written to the HTML.

